

# AssignmentOne

EXTRA

**DATA TYPES & ARRAYS VERIFICATION COURSE**

**BY: ENG. ISLAM HAMDY MOHAMED MOSTAFA**

**GUIDED BY: ENG. HASSAN KHALED**

# 1. Code Snippets

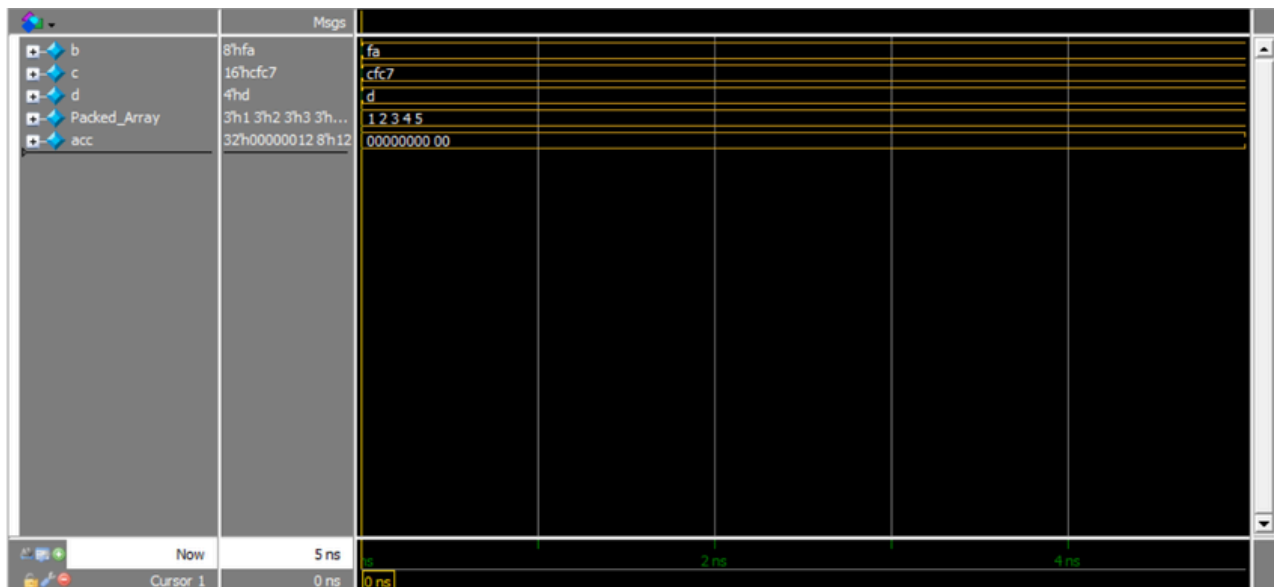
```
1  module AssignmentOne_Extra;
2
3      // never needs X/Z means 2 states and from 0 to 255 means 8 bits which equal 1 byte
4      byte unsigned b;
5      // from -32000 to 32000 means 16 bits using shortint 16 bits size and signed by default
6      shortint c;
7      // including X/Z means 4 states use logic
8      logic [3:0] d;
9
10
11     // fixed set of whole number weights means Fixed Array in order means Packed Array
12     bit [4:0] [2:0] Packed_Array = '{1, 2, 3, 4, 5};
13     // you never need to treat these weights as a single vector of adjacent elements means Unpacked Array
14     bit [2:0] Unpacked_Array [4:0] = '{1, 2, 3, 4, 5};
15
16     //Used an unpacked array of packed pairs to keep A/B and C/D
17     //internally contiguous while forcing a non-contiguous separation between the two pairs.
18     typedef bit [1:0][7:0] gain_pair_t;
19     gain_pair_t gain_table [2];
20
21     // The struct type reg_t is used to define a register with a name, address, and reset value.
22     // The regfile array is an array of reg_t structures that can hold multiple registers.
23     typedef struct {
24         string name;
25         byte unsigned address;
26         byte unsigned Reset_Value;
27     } reg_t;
28
29     reg_t regfile [0:2];
30
31     // I used a union type reg_view_u to allow access to the same data as either an integer or a byte.
32     typedef union {
33         int First;
34         bit [7:0] Second;
35     } reg_view_u;
36
37     reg_view_u acc;
38
39     //I used a dynamic array of 32-bit Logic values because it can change size during simulation and
40     //can hold a variable number of 32-bit values.
41     logic [31:0] trans_monitor [];
42
```

```

1  initial begin
2      $display("\n===== Part A =====");
3      b = 250;
4      $display("\nValue of b: %0d",b);
5      c = -12345;
6      $display("\nValue of c: %0d",c);
7      d = 4'b1101;
8      #5;
9      $display("\nValue of d: %0b",d);
10
11     $display("\n===== Part B =====");
12
13     $display("\n===== 1 =====");
14
15     $display("\nPacked Array : %p",Packed_Array);
16     $display("\nUnpacked Array: %p",Unpacked_Array);
17
18     $display("\n===== 2 =====");
19
20     gain_table[0] = '{10, 20};
21     gain_table[1] = '{30, 40};
22     $display("Gain Table [A,B],[C,D]: %p", gain_table);
23
24     $display("\n===== 3 =====");
25
26     regfile[0] = '{name:"CTRL" , address:8'h00 , Reset_Value:8'h01};
27     regfile[1] = '{name:"STATUS", address:8'h04 , Reset_Value:8'h00};
28     regfile[2] = '{name:"DATA" , address:8'h08 , Reset_Value:8'hFF};
29     $display("\nRegister File: %p",regfile);
30
31     $display("\n===== 4 =====");
32
33     acc.First = 32'h12345678;
34     $display("\nPrediction of byte value = 'h78");
35     $display("\nActual Output First: %0h, Actual Output Second: %0h", acc.First, acc.Second);
36
37     acc.First = 8'h12;
38     $display("\nActual Output First after change: %0h, Actual Output Second after change: %0h", acc.First, acc.Second);
39
40     $display("\n===== Part C =====");
41
42     trans_monitor = new[5];
43     trans_monitor = '{32'hAAAA_AAAA, 32'hBBBB_BBBB, 32'hCCCC_CCCC, 32'hDDDD_DDDD, 32'hEEEE_EEEE};
44     $display("Dynamic Array (Monitored Transactions): %p", trans_monitor);
45
46     $display("\n=====completed=====");
47
48     end
49
50
51 endmodule

```

## 2. Snippets for the waveform



## 3. Snippets for the logs

```

# ----- Part A -----
#
# Value of b: 250
#
# Value of c: -12345
#
# Value of d: 1101
#
# ----- Part B -----
#
# ----- 1 -----
#
# Packed Array : '{1, 2, 3, 4, 5}'
#
# Unpacked Array: '{1, 2, 3, 4, 5}'
#
# ----- 2 -----
#
# Gain Table [A,B],[C,D]: '{(10, 20), (30, 40)}'
#
# ----- 3 -----
#
# Register File: '{(name:"CTRL", address:0, Reset_Value:1), (name:"STATUS", address:4, Reset_Value:0), (name:"DATA", address:8, Reset_Value:255)}'
#
# ----- 4 -----
#
# Prediction of byte value = 'h78'
#
# Actual Output First: 12345678, Actual Output Second: 78
#
# Actual Output First after change: 12, Actual Output Second after change: 12
#
# ----- Part C -----
#
# Dynamic Array (Monitored Transactions): '{2863311530, 3149642683, 3435973836, 3722304989, 4008636142}'
#
# -----completed-----

```

## 4. Answers

### Part B Question 1:

Packed. > A packed array stores all elements as a single, contiguous block of bits in memory. Since the coefficients are read sequentially, this layout maximizes spatial locality, avoids simulator word-alignment padding, and minimizes access time.

### Part B Question 2:

Difference from Case 1 Memory Allocation: In Case 1 (Packed), all elements are stored as a single, contiguous block of bits in memory with no padding. In Case 2 (Unpacked), each element is stored independently, and the simulator typically pads each element to a standard machine word boundary for independent access. Access & Operations: In Case 1, the entire array can be manipulated or assigned as a single large bit-vector. In Case 2, because you never need to treat them as a single vector, the elements can only be accessed individually by their indices or via array assignment patterns.

### Part B Question 4d:

Explanation: Union (Overlapped) vs. Struct (Grouped)

Union Behavior (Item 5 - Overlapped Storage): All members share the exact same memory location. When you write 32'h12345678 to the 32-bit word view, it directly overwrites the shared memory space, meaning the byte view instantly changes to reflect the lower bits (8'h78). Modifying one member always alters the others.

Struct Behavior (Item 4 - Grouped Storage): Members are allocated side-by-side in separate memory locations. Each field (like name, address, and reset value) has its own independent address space. Writing to one field has absolutely no effect on the values of the other fields.